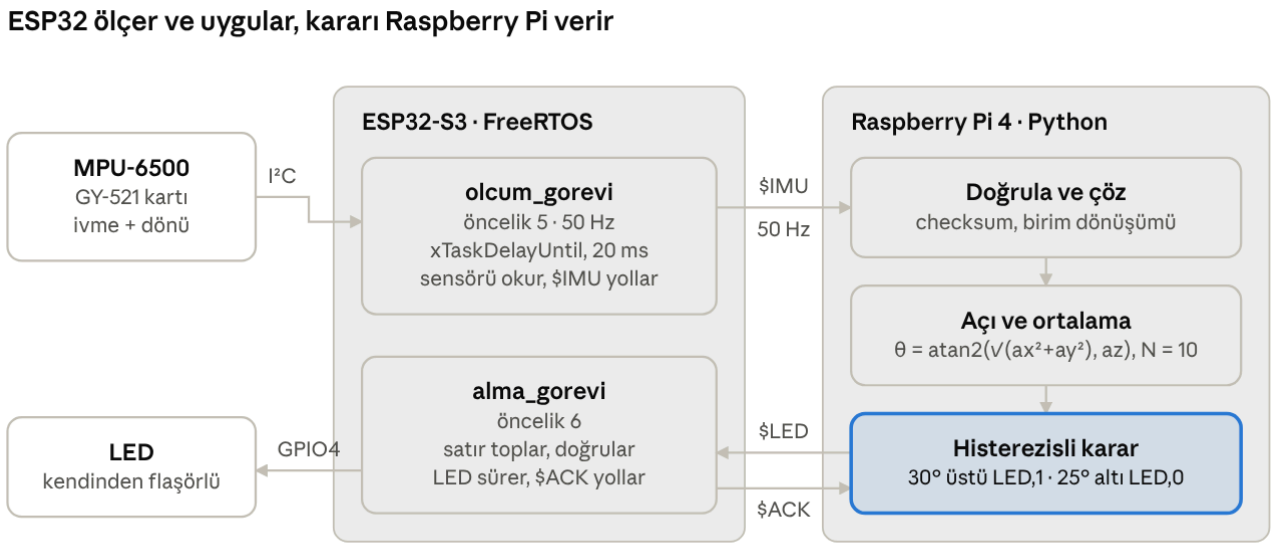
ESP32-S3 ile Raspberry Pi Haberleşmesi ve Eğim Alarmı

Ders: İleri Gömülü Sistemler (Yüksek Lisans) · **Vize Projesi Raporu** · 9 Kasım 2026
Öğrenci: Yasir YILDIRIM — 2610140003 · **Seçilen ekleme:** V1 — Eğim alarmı · **Depo:** `vize-2610140003`

1. Amaç

ESP32-S3, MPU-6050 uyumlu ivme/dönü sensörünü 50 Hz'de okur ve ölçümü UART üzerinden checksum'lı metin satırları olarak Raspberry Pi'ye gönderir; Pi veriyi doğrular, CSV'ye kaydeder ve ESP32'ye komut gönderir. Ekleme olarak seçilen **V1 eğim alarmı**, Pi'nin veriden karar vermesini sağlar: eğim 30°'yi geçince `$LED,1`, 25°'nin altına inince `$LED,0` gönderilir. Böylece ölçüm → karar → eylem döngüsü kapanır.

2. Mimari



ESP32 kesin zamanlama gerektiren işleri yapar: `olcum_gorevi` (öncelik 5) sensörü `xTaskDelayUntil` ile 20 ms periyotla okuyup `$IMU` satırı gönderir; daha yüksek öncelikli `alma_gorevi` (öncelik 6) UART'tan gelen komutları bekler, doğrular ve uygular. Karar Pi'de verilir; Pi her örneği ve her alarm olayını CSV dosyalarına yazar.

3. Bağlantı

ESP32-S3	Bağlandığı yer	Görevi	ESP32-S3	Bağlandığı yer	Görevi
GPIO17 (UART1 TX)	Pi pin 10 (RXD)	ESP32 → Pi veri	3V3	MPU VCC	Sensör beslemesi
GPIO18 (UART1 RX)	Pi pin 8 (TXD)	Pi → ESP32 komut	GPIO8 / GPIO9	MPU SDA / SCL	I²C, 400 kHz
GND	Breadboard GND hattı ← Pi pin 6, MPU GND	Ortak toprak	GPIO4	330 Ω → LED → GND	Alarm göstergesi

Kartlar ayrı USB kablolarından beslenir; Pi'nin 5V pinlerine bağlantı yapılmaz. Pi'de seri konsol kapatılıp UART açıldı (`raspi-config`).

4. Protokol

Her satır `$GOVDE*HH\r\n` biçimindedir; HH, `$` ile `*` arasındaki karakterlerin XOR'udur (örnek: `$LED,1*50`). 115200 baud, 8N1.

Yön	Mesaj	Açıklama
ESP32 → Pi	`$IMU,t,ax,ay,az,gx,gy,gz*HH`	t ms; ivme 16384 LSB/g, dönü 131 LSB/(°/s)

ESP32 → Pi	`$ACK,LED,0\|1` · `$ACK,HIZ,1..100`	Komut uygulandı
ESP32 → Pi	`$ERR,BICIM|CHECKSUM|KOMUT|I2C`	Hatalı satır, bilinmeyen komut, sensör okunamadı
Pi → ESP32	`$LED,0\|1*HH` · `$HIZ,1..100*HH`	LED; örnekleme hızı (Hz)

V1 protokole yeni mesaj eklemez: var olan `$LED` komutunu Pi'deki karara göre ve yalnızca durum değiştiğinde gönderir; her `$ACK`'in gecikmesi olay günlüğüne yazılır.

5. V1 eklemesi: eğim alarmı (`pi/egim_alarm.py`)

1. **Açı.** Hareketsiz sensörde ölçülen tek ivme yerçekimidir. Eğim, sensörün Z ekseni ile yerçekimi arasındaki açıdır (düz 0°, dik 90°, ters 180°) ve sensör hangi yöne eğilirse eğilsin doğrudur.
2. **Ortalama.** Son N = 10 örneğin (0,2 s) ivme *bileşenlerinin* ortalaması alınır, açı bu ortalama vektörden hesaplanır. Açıların kendisinin ortalaması alınmaz; ±180° sınırında anlamsız değer verir (Bölüm 7).
3. **Histerezis.** Alarm kapalıyken $\bar{\theta} > 30°$ → aç; açıkken $\bar{\theta} < 25°$ → kapat. 5°'lik bant, eşik çevresindeki titreşimi yutar.
4. **Yalnızca değişimde komut.** Hat saniyede 50 gereksiz komutla meşgul edilmez; her geçiş olay günlüğünde tek satırdır. Başlangıçta ve çıkışta LED söndürülür (güvenli durum).

$$\bar{a} = (1/N)\ \Sigma\ a_{k-i} \qquad \bar{\theta} = \text{atan2}(\ \sqrt{(\bar{a}_x^2 + \bar{a}_y^2)},\ \bar{a}_z\ ) \cdot 180/\pi$$

Yönergedeki $\theta = \text{atan2}(ax, az)$ formülü ilk sürümde kullanıldı ve `--tek-eksen` ile hâlâ seçilebilir; ancak yalnızca Y ekseni etrafındaki eğimi ölçtüğü için (Y yönünde 45° eğimde 0° verir) varsayılan yöntem değiştirildi. Eşikler ve pencere komut satırından ayarlanır (`--ust 30 --alt 25 --pencere 10`).

6. Ölçüm

Sistem 50 Hz'de kayıpsız çalıştı ve histerezis LED titremesini yaklaşık 6 kat azalttı. Gerçek sensörle alınan dört kayıttaki 10.153 satırdan yalnızca port açılırken yakalanan ilk yarım satır hatalıydı.

6.1 UART güvenilirliği ve örnek hızı

Kayıt	Süre	IMU satırı	Hatalı	Sonuç
60 sn kayıt, gerçek sensör (`kayit_60s.csv`)	60,4 s	3.021	0	50,0 örnek/s; aralık 20,0 ± 0,58 ms (19–21)
Diğer üç kayıt, gerçek sensör	142,5 s	7.132	1 (ilk yarım satır)	50,0 örnek/s
Uzun çalışma, sensörsüz (sentetik veri)	~3,4 dk	10.230	0	50,0 örnek/s

±1 ms'lik sapma ESP32 zaman damgasının milisaniyeye yuvarlanmasından gelir; `xTaskDelayUntil` periyodu kaymadan korudu.

6.2 `hiz` komutunun etkisi ve komut → ACK süresi

Komut	Yanıt	Beklenen	Ölçülen periyot (ort. ± ss)	n
`hiz 10`	`ACK,HIZ,10`	100 ms	100,0 ± 0,8 ms	152
`hiz 100`	`ACK,HIZ,100`	10 ms	10,0 ± 0,0 ms	1.230
`hiz 50`	`ACK,HIZ,50`	20 ms	20,0 ± 0,0 ms	605

Periyot ESP32 zaman damgasından hesaplandı. `alici.py`'nin gösterdiği “örnek/s” program başından beri ortalama olduğundan yeni hızı yavaş yansıttı (`hiz 10` sonrası 41,9 → 31,8 → … → 19,6).

`ack_olc.py`, 30 tekrar	Ortalama	Medyan	Std. sapma	Min	p95	Maks	Kayıp
`$LED` → `$ACK`	3,88 ms	3,38 ms	0,97 ms	3,36 ms	5,92 ms	5,96 ms	0 / 30

Taban süre ~3,4 ms: 11 baytlık komut ile 15 baytlık ACK 115200 baud'da birlikte ~2,3 ms sürer, kalanı işlemedir. Her 5–6 tekrarda bir süre ~5,9 ms'ye çıktı: ESP32 aynı UART'tan 20 ms'de bir ~44 baytlık (~3,8 ms) IMU satırı gönderiyor ve ACK o an giden satırın bitmesini bekliyor. Tekrarlar ~203,4 ms arayla geldiğinden komut IMU döngüsüne göre her seferinde 3,4 ms kayıyor ve ≈ 20/3,4 ≈ 6 tekrarda bir çakışıyor. Eğim alarmı çalışırken gönderilen 140'tan fazla `$LED` komutunun hepsi 3,3–3,4 ms'de onay aldı.

6.3 Açı doğruluğu ve histerezis

Konum	Gerçek	Ölçülen $\bar{\theta}$	Hata		Histerezis deneyi (aynı el hareketi, eşik çevresinde)	Süre	LED değişimi
Masada düz	0°	1,2°	+1,2°		Histerezissiz, ortalamasız (`--ust 30 --alt 30 --pencere 1`)	32,8 s	**112** (3,4/s)
Kitaba dik yaslı	90°	90,3° (88,7–91,3)	+0,3°		Histerezisli + 10 örnek ortalama (varsayılan)	35,5 s	**20** (0,56/s)

Alarm 30,3°'de açıldı, 24,1°'de kapandı. Hareketsiz sensörde |a| = 1,06–1,07 g okundu (beklenen 1,00 g): klon yongada Z ekseninde ~%6–7 ölçek/ofset hatası vardır; açı bileşenlerin oranından hesaplandığından etkisi küçük kaldı. Histerezissiz çalışmada 30,2° → 29,9° gibi 0,3°'lik titreşimler bile LED'i değiştirdi. Hızlı harekette sensör hareketin kendi ivmesini de ölçtüğü için açı anlık sıçradı; ortalama bunu azalttı ama tamamen bastıramadı (sınırlılık).

7. Sorunlar

Belirti	Teşhis	Çözüm
`probe device timeout`, “Sensör yok”; sensör kartının LED'i sönük	NACK değil zaman aşımı: GY-521'in I²C çekme dirençleri kartın üzerinde, kart beslenmeyince hatlar boşta. Kart bükülünce LED yanıyordu: pin şeridi lehimle bağlanmamıştı	VCC, GND, SCL, SDA yeniden lehimlendi
Saniyede 50 yerine ~263 satır; ESP32 zaman damgası satır başına ~3,8 ms arttı	Temas gidip gelirken her başarısız I²C okuması 100 ms zaman aşımı bekledi; periyot 20 ms olduğundan görev her seferinde takvimin 80 ms gerisine düştü. Temas düzelince `xTaskDelayUntil` beklemeden dönüp birikmiş örnekleri UART'ın en yüksek hızında (115200 ÷ 10 ÷ ~44 ≈ 262 satır/s) gönderdi	RST ile takvim sıfırlandı, temas giderildi. `vTaskDelay` ile patlama olmazdı, ama periyot okuma süresi kadar kayardı
Veri düzenli sinüs, az hep 16384, jiroskop 0	Firmware açılışta sensörü bulamayınca sentetik veri üretiyor; GND kablosu taşınırken temas kopmuştu	Bağlantı düzeltildi, RST
`WHO_AM_I = 0x70` (beklenen 0x68)	0x70 MPU-6500'ün kimliği; kullanılan registerlar (0x1A–0x1C, 0x3B–0x48, 0x6B) ve ölçekler aynı	Firmware değişmedi
Sensör ters çevrilince iki yanlış alarm ($\bar{\theta}$ = +30,4°, +35,6°)	Açı sarılması: ilk sürüm açıların ortalamasını alıyordu; +179° ile −174°'nin düz ortalaması anlamsız ara değer verdi	Bileşen ortalamasına geçildi; tekrar denemede yanlış alarm 0
Elle eğince θ bazen değişmedi ya da eksiye düştü	atan2(ax, az) yalnızca tek eksendeki eğimi ölçüyor	Varsayılan atan2(√(ax²+ay²), az) yapıldı
`idf.py set-target` hatası: `gömülü` → `g├Âm├╝l├╝`	ESP-IDF araçları Türkçe karakterli ve boşluklu yolu bozuyor; `subst` kısayolu da işe yaramadı (araç gerçek yolu kullanıyor)	Proje `C:\esp32_vize`'ye kopyalandı
Yükleme: “No serial data received”, ardından COM3 kayboldu	Yerleşik USB-Serial/JTAG girişinde otomatik yükleme modu çalışmadı; elle geçişte COM numarası 3'ten 4'e değişti	BOOT + RST, yeni COM ile yüklendi

8. Sonuç

UART haberleşmesi 50 Hz'de hatasız çalıştı ve iki yönlü komut zinciri (`$LED` → `$ACK`, medyan 3,38 ms) doğrulandı. V1 eklemesiyle Pi, her yönde eğimi ölçüp histerezisli eşikle ESP32'deki LED'i süren bir ölçüm → karar → eylem döngüsü kurdu; histerezis titremeyi 112'den 20 değişime indirdi. En öğretici bulgu, bir donanım temas sorununun yazılımda beklenmedik bir zamanlama etkisine (`xTaskDelayUntil` yetişme patlaması) dönüşebilmesiydi.

Yapay zekâ kullanım beyanı. Bu çalışmada Claude (Anthropic) yapay zekâ asistanından; kurulum adımlarının planlanması, arıza teşhisi, `egim_alarm.py` ve `ack_olc.py` betiklerinin yazımı ile README ve rapor taslağının hazırlanmasında yardım alındı. Donanım kurulumu, lehim, tüm ölçümler ve testler tarafımdan yapıldı; kullanılan kodu satır satır inceledim ve açıklayabilirim. ESP32 firmware'i ile `protokol.py` ve `alici.py` dersin şablonundan değiştirilmeden alındı.